Bài tập lớn A1: CNN và Transformer

CO5085 Học sâu và ứng dụng trong thị giác máy tính, học kỳ 261 (2026–2027)

Huỳnh Văn Tâm Thiện (2470094)
Lê Hữu Trọng (2470578)

Giảng viên: TS. Lê Thành Sách, TS. Nguyễn Quốc Minh
Trường Đại học Bách Khoa, ĐHQG-HCM

21/10/2026

Mục lục

1 Tóm tắt và đối chiếu với yêu cầu của đề **2**

2 Mục tiêu **3**

3 Tập dữ liệu **4**

4 Chia train, validation, test **5**

5 Khảo sát dữ liệu **6**

6 Tiền xử lý và tăng cường dữ liệu **8**

7 Mô hình **9**

8 Chế độ huấn luyện **11**

9 Giao thức thí nghiệm **12**

10 Chiến lược đánh giá **13**

11 Phần cứng và ngân sách **14**

1 Tóm tắt và đối chiếu với yêu cầu của đề

Báo cáo A1.1: chốt tập dữ liệu, hai mô hình và giao thức thí nghiệm của bài tập lớn A1. A1 so sánh một mạng tích chập với một Transformer thị giác trên cùng một tập ảnh lớn, mỗi mô hình ở bốn chế độ huấn luyện.

- **Tập dữ liệu.** Food-101, 101 lớp món ăn. Sau khi tách train, validation và test có 68 175, 7 575 và 25 250 ảnh (Mục 4).
- **Mô hình.** ResNet-50 (`microsoft/resnet-50`) cho CNN và DeiT-S (`facebook/deit-small-patch16-224`) cho Transformer, cả hai là bản công bố trên Hugging Face, có trọng số ImageNet-1k. Chế độ scratch không nạp trọng số đó (Mục 7).
- **Chế độ huấn luyện.** Train từ đầu (*scratch*), chỉ train đầu phân loại (*head*), đóng băng nửa đầu mạng (*partial*) và finetune toàn bộ (*full*). Hai mô hình nhân bốn chế độ thành 8 run, seed 42 (Mục 8, 9).
- **Tiền xử lý.** Một pipeline tăng cường và chuẩn hoá, dùng chung cho mọi mô hình và mọi chế độ (Mục 6).

Hạng mục	Liên kết	Nội dung được dùng
Food-101	huggingface.co/datasets/ethz/food101	Thẻ mô tả do nhóm ETH Zürich đăng: 101 lớp, 750 ảnh train và 250 ảnh test mỗi lớp
Điều khoản sử dụng	huggingface.co/datasets/ethz/food101#license-agreement	Không có giấy phép mở. Dữ liệu chỉ được dùng để học và nghiên cứu
ResNet-50	huggingface.co/microsoft/resnet-50	ResNet v1.5 (bước nhảy 2 ở tích chập 3×3), pretrain trên ImageNet-1k ở 224×224; thẻ mô tả không nêu công thức huấn luyện hay accuracy (Mục 7)
DeiT-S	huggingface.co/facebook/deit-small-patch16-224	DeiT-small không chưng cất, pretrain và finetune trên ImageNet-1k ở 224×224, top-1 79.9% theo thẻ mô tả

Bảng 1: Nguồn của tập dữ liệu, điều khoản sử dụng và hai bộ trọng số.

Yêu cầu của đề	Cách báo cáo đáp ứng	Mục
Tập ảnh công khai, ít nhất 10 lớp, vài nghìn ảnh train; không dùng MNIST, Fashion-MNIST hay CIFAR-10	Food-101 có 101 lớp và 68 175 ảnh train, công khai, không thuộc ba tập bị loại	3
Nêu giấy phép	Không có giấy phép mở; ảnh thuộc Foodspotting và chỉ được dùng cho học tập, nghiên cứu	Bảng 1
Nêu lý do chọn tập	Bốn tiêu chí chọn, kèm các tập đã loại và lý do loại	3
Mô tả cách chia tập	Giữ test chính thức; validation là 10% train, phân tầng theo lớp, seed 42; mọi run dùng chung split	4
Mô tả augmentation	Một pipeline lúc train, một pipeline lúc đánh giá, và cách chuẩn hoá	6
Nêu CNN và Transformer sẽ train	ResNet-50 và DeiT-S: kiến trúc, số tham số và chi phí tính toán	7
Kế hoạch train từ đầu, dùng pre-trained và đóng băng	Bốn chế độ, phần mạng bị đóng băng, 8 run và siêu tham số	8, 9

Bảng 2: Đối chiếu yêu cầu của đề với nội dung báo cáo.

2 Mục tiêu

Bài tập A1 so sánh một mạng tích chập (CNN) và một Transformer thị giác (ViT) trên một tập ảnh đủ lớn và đủ khó để khác biệt giữa hai họ kiến trúc, và giữa các cách dùng trọng số pretrained, có ý nghĩa. Bài chọn **Food-101** (101 món ăn, 101 000 ảnh), **ResNet-50** cho họ CNN và **DeiT-S** cho họ Transformer. Mỗi mô hình được huấn luyện theo bốn chế độ: từ đầu (*scratch*), và ba cách dùng trọng số pretrained ImageNet-1k là chỉ train đầu phân loại (*head*), đóng băng nửa đầu mạng (*partial*) và finetune toàn bộ (*full*).

Từ đó, báo cáo A1.2 sẽ so sánh bốn nội dung: (1) CNN và ViT khi train từ đầu; (2) lợi ích của pretrain đối với mỗi họ; (3) mức accuracy mất đi khi đóng băng một phần hoặc toàn bộ mạng; (4) chi phí tính toán của từng cấu hình (thời gian, bộ nhớ, số tham số được học, tốc độ suy luận).

3 Tập dữ liệu

Food-101 gồm 101 lớp món ăn, mỗi lớp 1 000 ảnh (750 ảnh train và 250 ảnh test theo split chính thức), cạnh dài nhất của ảnh không quá 512 px. Ảnh test đã được kiểm tra thủ công; ảnh train cố ý để nguyên nên còn nhiễu, chủ yếu là màu quá gắt và đôi khi sai nhãn.

Food-101 được chọn vì bốn lý do:

- **Đủ lớn và đủ khó.** 101 lớp, 75 750 ảnh train chính thức; bài toán phân loại chi tiết, nhiều món khác nhau trông gần giống nhau.
- **Không trùng với ImageNet.** Tiny-ImageNet, Imagenette và các tập con của ImageNet bị loại vì ảnh của chúng nằm trong dữ liệu mà mô hình pretrained đã học, làm lệch so sánh scratch với pretrained.
- **Hợp với mô hình pretrained.** Ảnh tự nhiên, dùng được ở độ phân giải 224 px; CIFAR-100 (32×32) phải phóng to gấp 7 lần nên bị loại.
- **Vừa ngân sách.** Khoảng 68 000 ảnh train đủ nhỏ để train từ đầu trên một GPU L4 (Mục 11).

Lưu ý: ImageNet-1k có sẵn một số lớp món ăn trùng hoặc gần với Food-101, nên trọng số pretrained có lợi thế ngữ nghĩa ở các lớp này; A1.2 sẽ xem accuracy từng lớp có phản ánh điều đó không.

4 Chia train, validation, test

Split test chính thức được giữ nguyên. Validation được tách từ split train chính thức: 10% mỗi lớp, phân tầng theo nhãn, seed 42 (Bảng 3). Split này cố định cho mọi run, ở cả hai mô hình và cả bốn chế độ, nên scratch và pretrained được so sánh trên cùng một split.

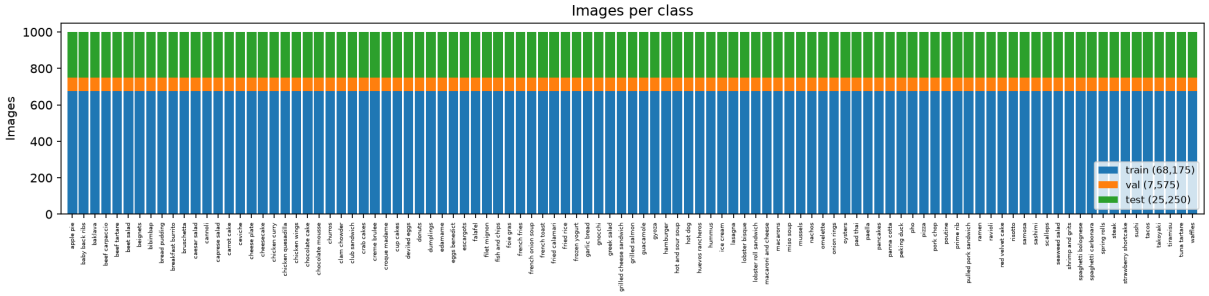
Tập	Số ảnh	Ảnh/lớp	Nguồn
train	68 175	675	90% split train chính thức
validation	7 575	75	10% split train chính thức
test	25 250	250	split test chính thức

Bảng 3: Ba tập dùng trong A1.

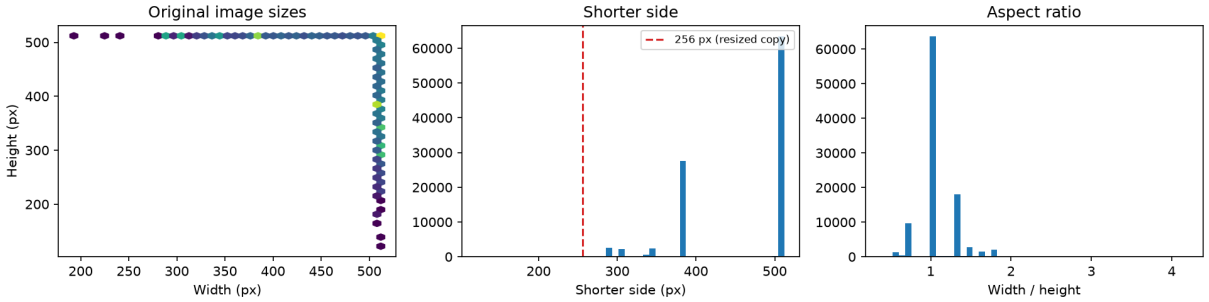
- **Test chỉ dùng một lần,** sau khi train xong; mọi quyết định (epoch giữ lại, learning rate) dựa trên validation.
- **Validation lấy từ train, không từ test,** để test không tham gia chọn mô hình.
- **Nhiễu nhãn:** validation lấy từ train nên cũng nhiễu, còn test đã được kiểm tra thủ công; validation accuracy vì vậy có thể thấp hơn test accuracy.

5 Khảo sát dữ liệu

Kích thước ảnh được đo trên toàn bộ 101 000 ảnh gốc, trước khi thu nhỏ; mean và std theo kênh được tính trên toàn bộ 68 175 ảnh train, sau bước cắt cố định (Hình 1–3, Bảng 4).



Hình 1: Số ảnh mỗi lớp trong train, validation và test.



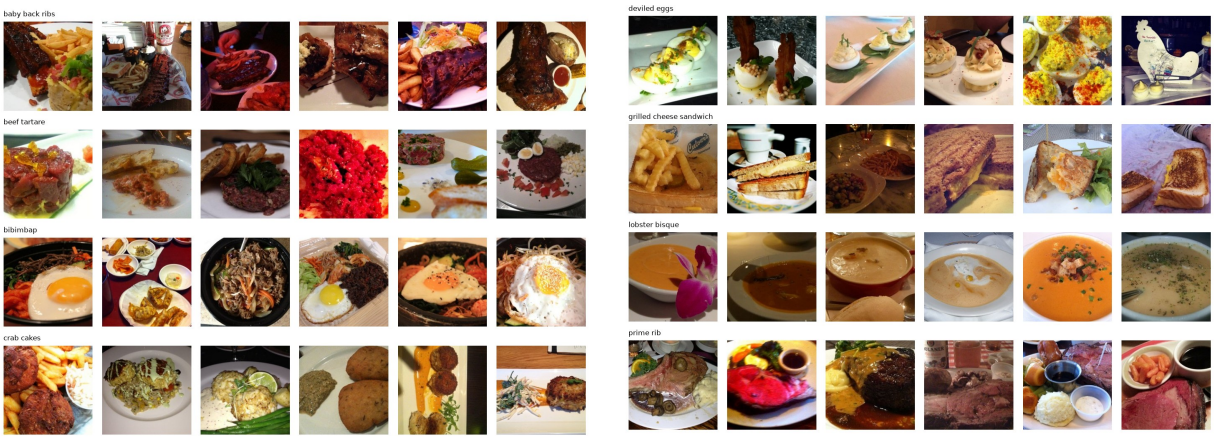
Hình 2: Kích thước ảnh gốc: phân bố rộng–cao, cạnh ngắn (vạch đỏ là 256 px của bản đã thu nhỏ) và tỉ lệ khung.

Đại lượng	Giá trị
Ảnh gốc	101 000 (L 3, RGB 100 997)
Cạnh dài (px)	512–512, trung vị 512
Cạnh ngắn (px)	122–512, trung vị 512
Cạnh ngắn < 256 px	44 ảnh
Tỉ lệ rộng/cao	0.38–4.20, trung vị 1.00
Mean RGB (68 175 ảnh train)	(0.557, 0.442, 0.327)
Std RGB	(0.260, 0.264, 0.266)

Bảng 4: Thống kê ảnh gốc và mean/std theo kênh RGB.

Nhận xét.

- **Cân bằng lớp.** Mỗi lớp có đúng 675 ảnh train, 75 ảnh validation và 250 ảnh test (Hình 1), nên không cần trọng số lớp.
- **Kích thước.** Cạnh dài luôn là 512 px. Theo Hình 2, khoảng 63% ảnh là ảnh vuông 512×512 và khoảng 27% có cạnh ngắn 384 px (khung 4:3); các tỉ lệ này đọc từ biểu đồ nên chỉ xấp xỉ. Chỉ 44 ảnh (0.04%) có cạnh ngắn dưới 256 px, được phóng lên khi đổi cạnh ngắn về 256 px, và quá ít để ảnh hưởng kết quả. Phép cắt giữa bỏ phần rìa của ảnh, đáng kể nhất với ảnh có khung dài (tỉ lệ đến 4.2), nhưng các ảnh này rất hiếm.
- **Chế độ màu.** 100 997 ảnh RGB và 3 ảnh xám (chuyển sang RGB khi tiền xử lý).
- **Màu sắc.** Mean RGB là $(0.557, 0.442, 0.327)$, std khoảng 0.26: ảnh món ăn nghiêng về đỏ hơn ImageNet $(0.485, 0.456, 0.406)$. Sau chuẩn hoá mean = std = 0.5, std chỉ còn khoảng 0.52, nhỏ hơn hai lần so với lúc pretrain (Mục 6).



Hình 3: Ảnh train của một số lớp, sau bước cắt cố định. Bên trái: bốn lớp đầu; bên phải: bốn lớp sau.

- **Nhiễu nhãn** (Hình 3). Ảnh chụp trong nhà hàng nên ánh sáng, góc chụp và nền rất khác nhau. Trong 48 ảnh mẫu có ít nhất hai ảnh mà nhãn không ứng với chủ thể rõ ràng (ảnh thứ hai của *bibimbap* chủ yếu là há cảo; ảnh thứ ba của *grilled cheese sandwich* không thấy bánh) và vài ảnh có màu rất gắt, đúng như tác giả mô tả. Mẫu quá nhỏ để ước lượng tỉ lệ nhiễu.
- **Cặp lớp dễ nhầm**: các món gần nhau như *prime rib*, *steak*; sẽ được đo ở A1.2 bằng ma trận nhầm lẫn (Mục 10).

6 Tiền xử lý và tăng cường dữ liệu

Mọi mô hình và mọi chế độ dùng chung một pipeline dữ liệu, giống MNIST ở E1 và E2: các run chỉ khác nhau ở mô hình và cách huấn luyện, không khác ở dữ liệu đưa vào.

Cắt cố định, áp dụng một lần cho mọi ảnh. Mọi ảnh của cả ba tập (train, validation, test) được xử lý đúng một lần theo cùng một cách: chuyển sang RGB, đổi cạnh ngắn về 256 px (giữ tỉ lệ, nội suy bicubic; ảnh nhỏ hơn được phóng lên), rồi cắt phần giữa 224×224 và lưu lại dạng JPEG.

Sau bước đó, chỉ tập train được tăng cường dữ liệu (Bảng 5); validation và test chỉ được chuẩn hoá.

Bước	Train	Validation, test và tập train dùng để đánh giá
Cắt cố định (một lần, mọi ảnh)	Đổi cạnh ngắn về 256 px, cắt giữa 224×224	như train
Lật	Lật ngang với xác suất 0.5	–
Xoay	Xoay ngẫu nhiên trong khoảng $\pm 15°$	–
Màu	Độ sáng, tương phản, độ bão hoà đổi trong $\pm 20\%$; sắc độ xoay trong ± 0.1 vòng màu ($\pm 36°$)	–
Dịch	Dịch ngẫu nhiên tối đa 10% theo mỗi chiều	–
Chuẩn hoá	Đưa về tensor trong $[0, 1]$, rồi trừ mean 0.5 và chia std 0.5 trên từng kênh	như train

Bảng 5: Pipeline dữ liệu dùng chung, theo đúng thứ tự áp dụng.



Hình 4: Cột đầu: ảnh sau bước cắt cố định. Các cột sau: cùng ảnh qua các phép tăng cường của pipeline train sáu lần.

7 Mô hình

Hai mô hình dùng trọng số công bố trên Hugging Face (Bảng 1), có số tham số và chi phí tính toán gần nhau, và cùng được pretrain chỉ trên ImageNet-1k để nguồn pretrain không làm lệch so sánh (Bảng 6). Đầu phân loại được thay bằng một lớp tuyến tính 101 lớp, khởi tạo ngẫu nhiên. Số tham số và kích thước đầu ra dưới đây được đo trực tiếp trên mô hình.

Họ	Mã trên Hugging Face	Tham số (đầu 1000 lớp)	Tham số (đầu 101 lớp)	GMAC/ảnh	Pretrain
CNN	`microsoft/resnet-50`	25 557 032	23 714 981	4.09	ImageNet-1k
Transformer	`facebook/deit-small-patch16-224`	22 050 664	21 704 549	4.60	ImageNet-1k

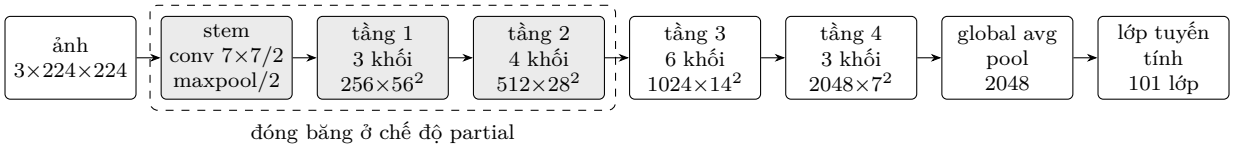
Bảng 6: Hai mô hình. Số tham số của bản Hugging Face (đầu phân loại 1000 lớp của ImageNet) và của mô hình dùng trong bài (đầu 101 lớp cho Food-101); hai số chênh nhau đúng bằng phần chênh của đầu phân loại. MAC cho một ảnh 224×224, đếm bằng bộ đếm phép tính của PyTorch với attention được tính bằng các phép nhân ma trận riêng lẻ: bộ đếm không ghi nhận kernel attention gộp, nên nếu dùng kernel đó sẽ thiếu khoảng 0.36 GMAC của DeiT-S.

7.1 ResNet-50

ResNet-50 gồm một stem, bốn tầng khối *bottleneck* có kết nối tắt, global average pooling và một lớp tuyến tính (Bảng 7, Hình 5). Mỗi khối bottleneck gồm ba tích chập (1×1 thu số kênh, 3×3, rồi 1×1 mở rộng số kênh gấp 4), mỗi tích chập kèm BatchNorm; hai tích chập đầu có ReLU ngay sau, còn ReLU thứ ba đặt sau phép cộng với kết nối tắt. Kết nối tắt là ánh xạ đồng nhất, trừ khối đầu của mỗi tầng, nơi nó là một tích chập 1×1 kèm BatchNorm: ở tầng 1 chỉ để khớp số kênh (64 lên 256), ở tầng 2–4 để khớp cả số kênh lẫn độ phân giải. Ở khối đầu của tầng 2–4, bước nhảy 2 đặt ở tích chập 3×3 (biến thể ResNet v1.5, cũng là biến thể của bản Microsoft); tầng 1 giữ nguyên độ phân giải vì stem đã giảm bằng maxpool.

Tầng	Cấu hình	Đầu ra ($C \times H \times W$)	Tham số
stem	conv 7×7, 64, stride 2; maxpool 3×3, stride 2	$64 \times 56 \times 56$	9 536
tầng 1	$[1{\times}1, 64;\ 3{\times}3, 64;\ 1{\times}1, 256] \times 3$	$256 \times 56 \times 56$	215 808
tầng 2	$[1{\times}1, 128;\ 3{\times}3, 128;\ 1{\times}1, 512] \times 4$	$512 \times 28 \times 28$	1 219 584
tầng 3	$[1{\times}1, 256;\ 3{\times}3, 256;\ 1{\times}1, 1024] \times 6$	$1024 \times 14 \times 14$	7 098 368
tầng 4	$[1{\times}1, 512;\ 3{\times}3, 512;\ 1{\times}1, 2048] \times 3$	$2048 \times 7 \times 7$	14 964 736
head	global average pooling; lớp tuyến tính $2048 \to 101$	101	206 949
tổng			23 714 981

Bảng 7: Kiến trúc ResNet-50. Mỗi dòng "$[\cdot] \times n$" là n khối bottleneck; bước nhảy 2 ở khối đầu của tầng 2–4. Phần tô xám là phần bị đóng băng ở chế độ partial.



Hình 5: Luồng dữ liệu qua ResNet-50: độ phân giải giảm từ 224 xuống 7 trong khi số kênh tăng từ 64 lên 2048.

Tham số dồn về cuối mạng (tầng 3 và 4 chiếm 93.0%, stem đến tầng 2 chỉ 6.1%), trong khi MAC chia khá đều giữa các tầng (stem và tầng 1–4 lần lượt chiếm 2.9, 16.3, 25.1, 35.8 và 19.8%); nửa đầu mạng vì vậy chiếm 44% MAC nhưng chỉ 6.1% tham số. Thẻ mô tả của Microsoft không nêu công thức huấn luyện hay accuracy. Bộ trọng số này cho đầu ra giống hệt bản `resnet50.a1_in1k` của thư viện timm (đã đối chiếu trực tiếp), vốn được train bằng công thức mạnh của timm và

có top-1 trên ImageNet-1k ở 224×224 là 80.38%; con số này gần với 79.9% của DeiT-S nên hai mô hình được pretrain ở chất lượng tương đương.

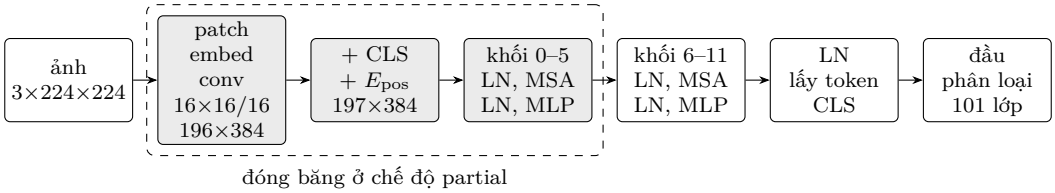
7.2 DeiT-S

DeiT-S dùng kiến trúc Vision Transformer ở cấu hình nhỏ (patch 16, 12 khối, 384 chiều, 6 head); điểm mới của DeiT nằm ở công thức huấn luyện, giúp ViT đạt accuracy tốt khi chỉ train trên ImageNet-1k. Ảnh được cắt thành 196 patch 16×16, mỗi patch được chiếu tuyến tính lên một vector 384 chiều; một token CLS và positional embedding học được được thêm vào, rồi chuỗi 197 token đi qua 12 khối Transformer. Mỗi khối gồm hai phần, mỗi phần có LayerNorm đứng trước và kết nối tắt: multi-head self-attention (6 head, mỗi head 64 chiều) và MLP ($384 \to 1536 \to 384$, hàm kích hoạt GELU). Đầu phân loại đọc token CLS sau khối cuối.

Thành phần	Cấu hình	Đầu ra ($N \times D$)	Tham số
patch embedding	conv 16×16, stride 16, $3 \to 384$	196×384	295 296
CLS, positional emb.	token 1×384; $E_{\text{pos}} \in \mathbb{R}^{197 \times 384}$	197×384	76 032
khối 0–5	$6\times$ [LN; MSA 6 head $\times$ 64; LN; MLP $384 \to 1536 \to 384$]	197×384	10 646 784
khối 6–11	như trên	197×384	10 646 784
head	LN; lớp tuyến tính $384 \to 101$ trên token CLS	101	39 653
tổng			21 704 549

Bảng 8: Kiến trúc DeiT-S. Phần tô xám là phần bị đóng băng ở chế độ partial.

Mỗi khối có 1 774 464 tham số (MSA 591 360, MLP 1 181 568, hai LayerNorm 1 536). Với $N = 197$, hai phép nhân ma trận của attention chỉ chiếm khoảng 7.9% MAC của một khối; phần lớn chi phí nằm ở các phép chiếu và MLP.



Hình 6: Luồng dữ liệu qua DeiT-S: 197 token $\times$ 384 chiều được giữ nguyên qua cả 12 khối.

Theo thẻ mô tả của Facebook, đây là bản DeiT-small không chưng cất, chỉ dùng dữ liệu ImageNet-1k ở 224×224, top-1 79.9%. Bản này cho đầu ra trùng với bản `fb_in1k` của thư viện timm đến sai số làm tròn (chênh lệch logit cỡ 10^{-5}, do hai cài đặt đặt hằng số epsilon của LayerNorm khác nhau).

8 Chế độ huấn luyện

Bảng 9 ghi định nghĩa bốn chế độ huấn luyện và số tham số được học của mỗi mô hình ở từng chế độ.

Chế độ	Pretrained	Đóng băng	ResNet-50 trainable	DeiT-S trainable
scratch	không	không	23 714 981 (100%)	21 704 549 (100%)
head	có	mọi thứ trừ head	206 949 (0.9%)	38 885 (0.2%)
partial	có	nửa đầu	22 270 053 (93.9%)	10 686 437 (49.2%)
full	có	không	23 714 981 (100%)	21 704 549 (100%)

Bảng 9: Bốn chế độ huấn luyện. "Nửa đầu" tính theo độ sâu: ResNet-50 đóng băng stem cùng tầng 1–tầng 2, DeiT-S đóng băng patch embedding, token CLS, positional embedding và khối 0–5.

Bất đối xứng của chế độ partial. Cùng là "đóng băng nửa đầu theo độ sâu", nhưng ResNet-50 vẫn học 93.9% tham số, vì tham số của nó dồn ở tầng 3 và tầng 4, trong khi DeiT-S chỉ học 49.2%, vì 12 khối của nó có cùng kích thước. Chúng tôi giữ định nghĩa theo độ sâu, vì nó ứng với câu hỏi "giữ nguyên các đặc trưng cấp thấp", và luôn báo cáo số tham số trainable bên cạnh kết quả để người đọc thấy rõ khác biệt này.

BatchNorm trong phần đóng băng. Đóng băng trọng số là chưa đủ: ở chế độ train, BatchNorm vẫn cập nhật trung bình và phương sai chạy. Vì vậy các lớp BatchNorm có toàn bộ tham số bị đóng băng được đặt lại về chế độ eval khi huấn luyện. Lần chạy thử cho thấy sau hai epoch ở chế độ head, mọi tensor của ResNet-50 ngoài lớp tuyến tính cuối, kể cả thống kê BatchNorm, giữ nguyên giá trị pretrained.

Weight decay và stochastic depth. Weight decay chỉ áp lên trọng số của các lớp tuyến tính và tích chập; bias, tham số chuẩn hoá, token CLS và positional embedding không bị decay. Stochastic depth (drop path) đặt 0.1 cho scratch và full, và 0 cho head và partial: bỏ ngẫu nhiên các khối đã đóng băng chỉ làm nhiễu đặc trưng cố định mà không giúp học thêm.

9 Giao thức thí nghiệm

Ma trận thí nghiệm gồm **8 run**: 2 mô hình × 4 chế độ, mỗi run train một lần với seed 42 trên toàn bộ tập train (Bảng 10). Cấu hình của từng run được sinh tự động từ một định nghĩa duy nhất, nên hai run chỉ khác nhau đúng ở những điểm nêu trong các bảng của báo cáo.

Chế độ	Epoch	LR đỉnh	Warmup (epoch)	Drop path	Clip gradient
scratch	40	5×10^{-4}	5	0.1	1.0
head	10	1×10^{-3}	1	0	–
partial	15	2×10^{-4}	1	0	–
full	15	1×10^{-4}	1	0.1	–

Bảng 10: Siêu tham số riêng theo chế độ, dùng cho cả hai mô hình. Learning rate là giá trị khởi đầu, sẽ được kiểm tra trên validation trước khi chạy chính thức.

Phần dùng chung cho mọi run: AdamW với weight decay 0.05, batch 128, cross-entropy với label smoothing 0.1, autocast bfloat16 trên GPU, và bố trí bộ nhớ theo thứ tự kênh cuối (*channels-last*) cho ResNet-50. Learning rate tăng tuyến tính trong W bước đầu rồi giảm theo cosine về 0, cập nhật sau mỗi bước tối ưu:

$$\eta_t = \begin{cases} \eta_{\max} \dfrac{t+1}{W}, & t < W, \\ \eta_{\max} \dfrac{1}{2}\left(1 + \cos \dfrac{\pi\,(t-W)}{T-W}\right), & t \geq W, \end{cases} \tag{1}$$

với T là tổng số bước (số epoch × 532 bước mỗi epoch) và W là số epoch warmup × 532.

Chọn checkpoint và dừng. Sau mỗi epoch, mô hình được đánh giá trên validation, và checkpoint có validation accuracy cao nhất được giữ lại. Không dùng early stopping: lịch cosine cần chạy hết để learning rate về 0. Một bản sao lưu đầy đủ (mô hình, optimizer, scheduler, trạng thái ngẫu nhiên) được ghi mỗi epoch, nên run bị ngắt có thể chạy tiếp đúng từ chỗ dừng.

Ngân sách epoch. Scratch được 40 epoch, các chế độ pretrained 10–15 epoch. So sánh trong A1 vì vậy là so sánh *dưới ngân sách cố định*, không phải giữa các mô hình đã hội tụ hoàn toàn; thời gian train được báo cáo cùng accuracy để thấy rõ đánh đổi.

Kiểm tra learning rate. Trước khi chạy chính thức, learning rate của scratch và full được thử ngắn (2–3 epoch) trên validation. Tập test không được dùng ở bước này. Mỗi lần thử là một run riêng, ghi vào thư mục riêng và không bao giờ được dùng làm run chính.

10 Chiến lược đánh giá

Mọi quyết định (chọn epoch, kiểm tra learning rate, chọn mô hình để phân tích lỗi) dựa trên validation; mỗi run giữ epoch có validation accuracy cao nhất và được đánh giá trên test đúng một lần, với cùng 25 250 ảnh test và không tăng cường dữ liệu. Đại lượng chính là test accuracy; ngoài ra báo cáo macro precision, recall, F1, accuracy từng lớp, chi phí (tham số trainable, thời gian mỗi epoch, bộ nhớ GPU đỉnh, tốc độ suy luận) và các cặp lớp bị nhầm nhiều nhất.

Mỗi cấu hình chỉ chạy một lần (seed 42), nên báo cáo chỉ định lượng được sai số do tập test hữu hạn: khoảng ± 0.44 điểm accuracy ở mức tin cậy 95% khi accuracy khoảng 85%. Hai run được so sánh theo cặp trên cùng các ảnh test, bằng khoảng tin cậy bootstrap và kiểm định McNemar có hiệu chỉnh Holm. Một chênh lệch chỉ được coi là rõ ràng khi khoảng tin cậy không chứa 0 và chênh lệch tối thiểu 1 điểm accuracy; chênh lệch nhỏ hơn chỉ được mô tả, không dùng làm luận cứ.

Chỉ mười một so sánh định trước ở Bảng 11 được kiểm định; các so sánh khác chỉ mô tả và được ghi rõ là phát sinh sau khi xem kết quả. Mỗi so sánh là hiệu accuracy trên test giữa hai run, riêng so sánh cuối là hiệu của hai hiệu. Chi phí (nội dung 4 ở Mục 2) không được kiểm định vì thời gian, bộ nhớ và số tham số là các đại lượng đo trực tiếp, không có sai số lấy mẫu trên tập test; chúng được báo cáo cạnh accuracy.

Nội dung	Phép so sánh	Số cặp
1. CNN so với ViT	DeiT-S so với ResNet-50, trong từng chế độ	4
2. Giá trị của pretrain	full so với scratch, riêng từng mô hình	2
	mức tăng nhờ pretrain của DeiT-S so với của ResNet-50	1
3. Giá của việc đóng băng	head và partial so với full, riêng từng mô hình	4
Tổng		11

Bảng 11: Mười một so sánh được kiểm định, thuộc ba nội dung đầu ở Mục 2.

Cần đọc kết quả kèm các hạn chế: scratch được nhiều epoch hơn (40 so với 10–15), nên khoảng cách đo được không phóng đại giá trị của pretrain; nhiễu nhãn khiến validation có thể thấp hơn test; và chuẩn hoá lệch lúc pretrain (Mục 6) có thể làm thấp kết quả của head và partial.

11 Phần cứng và ngân sách

Thí nghiệm chạy trên Modal, mỗi run là một container với một GPU NVIDIA L4 (24 GB), 8 nhân CPU và 16 GB RAM. Ngân sách tối đa là \$20, với đơn giá ước tính khoảng \$1.3–1.6 mỗi giờ cho GPU, CPU và bộ nhớ. Mốc A1.1 chưa có số đo tốc độ. Ước tính sơ bộ dựa trên giả định khoảng 800 ảnh/giây khi train (chưa kiểm chứng) là khoảng 4 GPU-giờ cho 8 run, tức khoảng \$6; nếu tốc độ thực chỉ bằng một nửa thì khoảng \$10–13, vẫn trong ngân sách. Trước khi chạy chính thức, tốc độ train, tốc độ suy luận và bộ nhớ đỉnh của từng mô hình ở từng chế độ sẽ được đo trên chính GPU đó, rồi số epoch ở Bảng 10 được chốt theo kết quả đo và ghi vào báo cáo A1.2.